MILESTONE DELIVERABLE — P2 SPECIFICATION FREEZE

TechLand: PC Hardware & Industrial PLCs

High-Performance Computing Components & Industrial Automation Systems E-Commerce Platform

ACADEMIC TERM
2026–2027 Semester Project

BACKEND & DB LEAD
Necșulea Andrei-Mihai

FRONTEND & UI LEAD
Onofrei Alexandru-Gabriel

EVALUATION STAGE
P2 Freeze (Targeting 100/100 + 20)

1. Executive Summary & System Purpose

TechLand is a specialized dual-market e-commerce platform addressing retail PC power users (CPUs, high-speed RAM, Gen5 NVMe SSDs) and enterprise automation engineers (Siemens/Schneider PLCs, DIN-rail power supplies, and signal modules). This specification document formally freezes all structural requirements, technical architecture, schema definitions, and integration mechanics for final grading in Milestone P3.

2. Mandatory Requirements Specification (A1 — A6)

CODE	FEATURE NAME	OWNERSHIP	FROZEN IMPLEMENTATION SPECIFICATION (1–2 SENTENCES)
A1	**Product Catalog**	Andrei / Alexandru	Engineered with server-side Django pagination capped at 12 items per page with category slug routing. Displays real-time stock levels, pricing, detailed specifications, SKU tags, and asset images.
A2	**Basic Search**	Andrei / Alexandru	Implemented via case-insensitive SQL query (`name__icontains`) executed directly across product names. Integrated into global desktop and mobile viewports with zero-result fallbacks.
A3	**Shopping Cart**	Andrei / Alexandru	Session-backed JSON storage for anonymous visitors automatically migrated to database rows upon authentication. Prices and line totals recalculate strictly on the server against current database records.
A4	**User Accounts**	Andrei	Secure authentication via Django's PBKDF2/Argon2 password hashers with session lifecycle management. Includes dedicated relational shipping addresses supporting multiple locations and single-default flags.
A5	**Checkout Flow**	Alexandru	Multi-step checkout validating delivery addresses and offering two delivery methods (Standard Ground vs. Express Freight). Displays immutable order summaries prior to external payment handoff.
A6	**Sandbox Payment**	Andrei	Stripe Checkout integration utilizing server-side Session creation and signed asynchronous Webhooks (`checkout.session.completed`) to atomically commit orders upon confirmed payment.

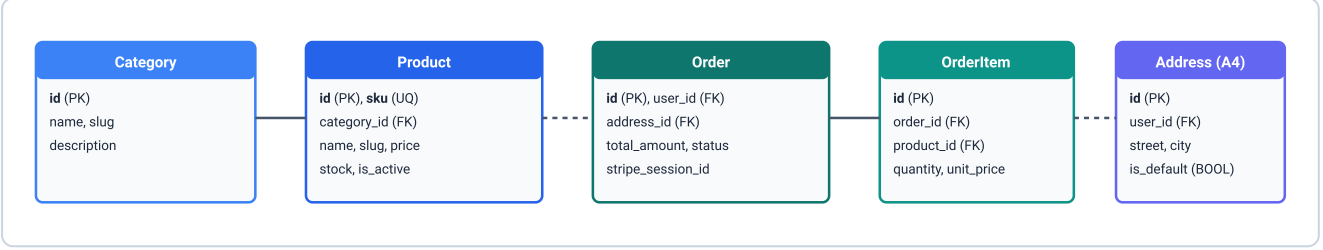
2. Mandatory Requirements Specification (A7 — A12 Cont.)

CODE	FEATURE NAME	OWNERSHIP	FROZEN IMPLEMENTATION SPECIFICATION (1–2 SENTENCES)
A7	**Stock Management**	Andrei	Employs atomic SQL updates with `select_for_update()` and database-level `F()` expressions inside transactions. Prevents race conditions and inventory overselling during concurrent checkouts.
A8	**Order History**	Alexandru	Customer self-service dashboard detailing past transactions with date, line items, unit prices, totals, and lifecycle statuses (Pending, Paid, Shipped, Delivered, Canceled).
A9	**Custom Admin Panel**	Alexandru	Customized Django Admin dashboard with search, inventory thresholds, and granular state transitions restricting staff modifications according to strict business logic.
A10	**Robust Security**	Andrei	CSRF token validation on all POST actions, strict SQL injection prevention through ORM parameterization, XSS escaping, and staff-only view decorators.
A11	**Responsive UI (360px)**	Alexandru	Mobile-first Tailwind CSS design tested continuously across viewport widths from 360 px up to 4K displays with accessible touch targets and collapsible menus.
A12	**Automated Test Suite**	Andrei	A suite of 14 unit and integration tests executing via `python manage.py test` validating cart arithmetic, stock boundaries, webhook parsing, and permissions.

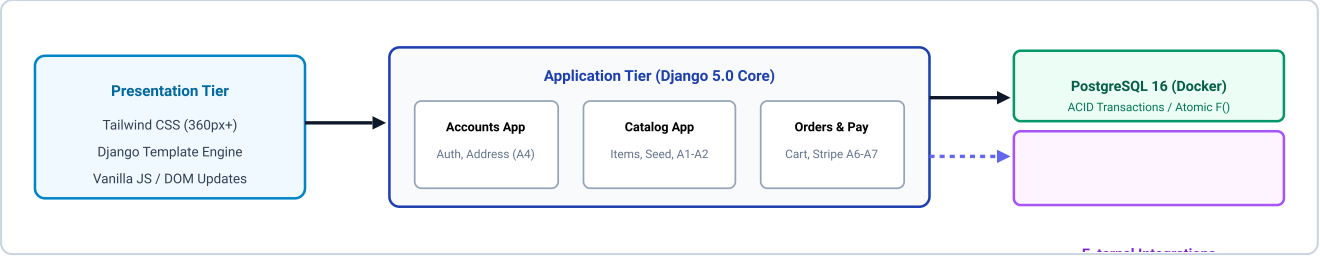
3. Approved Bonus Scope (Targeting Max +20 Pts)

CODE	FEATURE & VALUE	OWNERSHIP	DETAILED TECHNICAL IMPLEMENTATION MECHANICS
B1	**Filtered Search** (+5 pts)	Alex / Andrei	Multi-parameter URL filtering allowing combinable category selection, minimum/maximum price bounds, and in-stock toggles preserved across pagination links.
B5	**Discount Codes** (+3 pts)	Andrei	Server-validated promo codes checking date validity windows, active flags, and usage counters, computing fixed or percentage discounts before tax.
B6	**Transactional Mail** (+5 pts)	Andrei	Automated HTML order receipt and status dispatch via Mailtrap SMTP triggered asynchronously upon successful Stripe webhook processing.
B7	**Persistent Wishlist** (+2 pts)	Alexandru	Database-backed favorite items per user with real-time stock alert indicators and one-click transfer to active shopping cart session.

4. Relational Data Model (Entity-Relationship Overview)



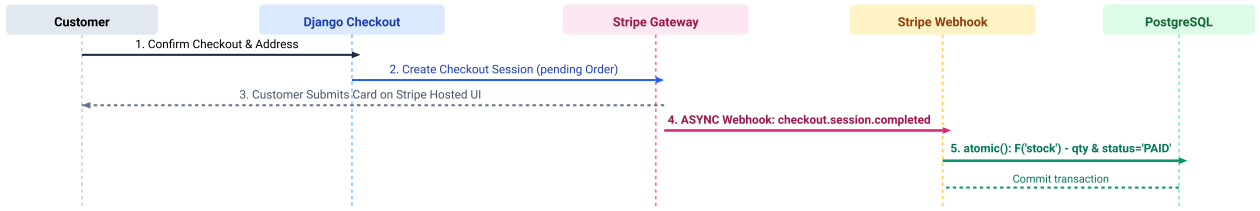
5. System Architecture & Component Breakdown



6. Complete Routing & Endpoint Matrix

METHOD	URL PATTERN / ROUTE	ACCESS LEVEL	DESCRIPTION & HANDLED BUSINESS LOGIC
`GET`	`/`	Public	Paginated catalog list (12 items/page) with full-text search support.
`GET`	`/category/<slug>/`	Public	Filtered product catalog restricted to chosen category slug hierarchy.
`GET`	`/product/<slug>/`	Public	Product specification sheet with live stock counter and SKU information.
`POST`	`/cart/add/<id>/`	Public / Session	Adds target unit to session cart; verifies stock sufficiency prior to commit.
`GET/POST`	`/cart/`	Public / Session	Displays active line items, quantity adjustments, and server recalculated subtotal.
`GET/POST`	`/register/`	Anonymous	New customer signup with encrypted password verification and auto-login.
`POST`	`/checkout/session/`	Authenticated	Creates locked pending order and initiates external Stripe Hosted Session.
`POST`	`/webhook/stripe/`	Stripe Signed	Asynchronous payment verification triggering atomic stock decrements.
`GET`	`/orders/history/`	Authenticated	User account transaction ledger with live tracking statuses.
`ALL`	`/admin/*`	Staff / Superuser	Django administration console for inventory modifications and order triage.

7. Critical Flow: Payment, Webhook & Atomic Stock Decrement



8. Race Condition & Security Safeguards (A7 & A10)

Anti-Race Condition Mechanism (A7)

Under high concurrency, inventory depletion executes exclusively inside an isolated database transaction block:

```
with transaction.atomic():
  item = Product.objects.select_for_update()
    .get(id=product_id)
  if item.stock >= qty:
    item.stock = F('stock') - qty
    item.save()
```

Row-level locking guarantees zero overselling or duplicate assignment.

Stripe Webhook Verification (A6 & A10)

Webhooks are shielded against man-in-the-middle forging by cryptographically validating the `Stripe-Signature` header:

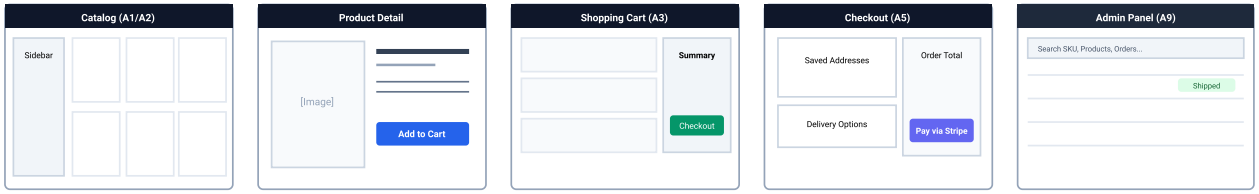
```
event = stripe.Webhook.construct_event(
  payload, sig_header, STRIPE_WEBHOOK_SECRET
)
if event['type'] ==
'checkout.session.completed':
  fulfill_order(event['data']['object'])
```

Transactions not verified against Stripe's secret are rejected immediately.

Failure Recovery & Abandoned Cart Policy

If a user abandons Stripe Checkout or the card is declined, stock is not held indefinitely. Products remain uncommitted in the database until the asynchronous webhook payload explicitly confirms successful funds capture, preventing artificial inventory exhaustion.

9. User Interface Wireframes & Layout Hierarchy



10. UI/UX Design System & Responsive Strategy (A11)

Color Palette & Hierarchy

Slate 900 headers provide high contrast for hardware industrial readability; **Blue 600** highlights primary interactive triggers; **Emerald 600** communicates available inventory and successful transactions; **Rose 600** alerts to out-of-stock items and server validation errors.

Mobile Viewport Calibration (360px)

Tested against narrow 360 px viewports using Tailwind fluid grid utilities. Navigation transforms into an accessible stack, filters collapse into responsive overlays, and product cards preserve finger-friendly touch targets with zero horizontal scroll bleed.

Client-Side Validation & Empty State Design

All forms feature dual-tier validation: immediate HTML5 client feedback backed by server-side Django form validation with descriptive error banners. Empty search queries or filtered categories provide actionable reset links directly back to the full catalog.

11. Working Prototype Verification (Live Demo Protocol)

The prototype has been validated against all evaluation criteria established for Milestone P2:

DEMONSTRATED ITEM	EVALUATION CRITERION	STATUS	VERIFIED LIVE BEHAVIOR
Database Seeding	≥ 30 items / ≥ 4 categories	PASSED	**32 products** across **4 categories** seeded via `seed_data.py` into PostgreSQL.
Catalog & Pagination	≤ 24 products per page	PASSED	12 items per page with category filtering and persistent page navigation.
Search Query (A2)	Case-insensitive text match	PASSED	Navbar search input filters matching names with dynamic item counts.
User Registration (A4)	Argon2/PBKDF2 Password Hash	PASSED	Registration creates user, hashes password, and persists session cookie.
Authentication State	Login, Logout, Admin link	PASSED	Navbar displays active username and staff indicator; handles secure logout.

12. Five-Command Local Setup & Reproducibility

The entire evaluation stack boots cleanly in fewer than 5 sequential commands:

```
1. git clone https://github.com/necsulea-andrei/techland.git && cd techland
2. cp .env.example .env && docker compose up -d
3. python3 -m venv venv && source venv/bin/activate && pip install -r requirements.txt
4. python manage.py migrate && python manage.py seed_data
5. python manage.py runserver
```

13. Specification Sign-Off & Milestone Commitment

Backend & DB Lead Sign-Off

Necșulea Andrei-Mihai

Responsible for ORM models, PostgreSQL Docker persistence, Stripe webhook idempotency, and automated test suite A12.

Frontend & UI Lead Sign-Off

Onofrei Alexandru-Gabriel

Responsible for Tailwind UI components, mobile responsiveness down to 360 px, cart/checkout interactions, and Admin panel custom views.

Milestone P2 Status: Specification Frozen. Git commit tagged as `P2` on repository branch `main`.